

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Aim

To install Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib, and verify the setup by running a simple Reinforcement Learning program.

Procedure

1. Install Anaconda Navigator and Python.
2. Open Anaconda Prompt.
3. Install the required libraries:
pip install tensorflow keras gymnasium numpy matplotlib
4. Create a Python file and import the required libraries.
5. Run a simple Gymnasium environment to verify the installation.
6. Ensure the program executes without errors and displays the initial environment state.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")
state, info = env.reset()

print("Initial State:", state)

env.close()
```

Output

Initial State: 0

Result

The Reinforcement Learning development environment was installed and configured successfully. All required libraries were imported without errors, and the Gymnasium environment executed correctly, confirming that the setup is ready for Reinforcement Learning experiments

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Aim

To create, initialize, and interact with Gymnasium environments such as FrozenLake, CartPole, and MountainCar to understand states, actions, rewards, and episodes.

Procedure

1. Import the Gymnasium library.
2. Create a Gymnasium environment (e.g., FrozenLake-v1).
3. Reset the environment to obtain the initial state.
4. Perform random actions using the action space.
5. Observe the state, reward, and episode status.
6. Close the environment after execution.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")
state, info = env.reset()

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)

env.close()
```

Output

State:	4	Reward:	0.0
State:	8	Reward:	0.0
State:	9	Reward:	0.0
State:	13	Reward:	0.0
State: 15 Reward: 1.0			

Result

The Gymnasium environment was successfully created and executed. The experiment demonstrated how an agent interacts with the environment through states, actions, rewards.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Aim

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Procedure

1. Import the Gymnasium library.
2. Create a Reinforcement Learning environment.
3. Initialize the environment and obtain the initial state.
4. Select random actions for the agent.
5. Observe the next state and reward after each action.
6. Repeat the process until the episode ends.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")
state, info = env.reset()

done = False

while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)
    done = terminated or truncated

env.close()
```

Output

State:	4	Reward:	0.0
State:	8	Reward:	0.0
State:	9	Reward:	0.0
State: 13 Reward: 0.0			

Result

The Reinforcement Learning framework was successfully implemented. The agent interacted with the environment by selecting actions, receiving rewards, and transitioning between states until the episode terminated.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Aim

To develop a simple Markov Decision Process (MDP) model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Procedure

1. Define the states, actions, and rewards.
2. Initialize the starting state.
3. Simulate state transitions based on selected actions.
4. Assign rewards for each transition.
5. Continue until the final state is reached.
6. Display the state transitions and rewards.

Program

```
states = ["A", "B", "Goal"]
transitions = {"A": "B", "B": "Goal"}
rewards = {"A": 0, "B": 1}
```

```
state = "A"
```

```
while state != "Goal":
    print("Current State:", state)
    print("Reward:", rewards[state])
    state = transitions[state]
```

```
print("Current State: Goal")
print("Process Completed")
```

Output

Current	State:	A
Reward:		0
Current	State:	B
Reward:		1
Current	State:	Goal

Result

The Markov Decision Process was successfully simulated. The experiment demonstrated state transitions, reward assignment, and policy execution, helping to understand sequential decision-making in Reinforcement Learning.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Aim

To implement the Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Procedure

1. Define the rewards and discount factor.
2. Initialize the state-value function.
3. Apply the Bellman update equation iteratively.
4. Update the value estimates until convergence.
5. Display the final state values.
6. Analyze the convergence of the value function.

Program

```
gamma = 0.9
reward = [0, 0, 1]
value = [0, 0, 0]

for _ in range(10):
    value[0] = reward[0] + gamma * value[1]
    value[1] = reward[1] + gamma * value[2]
    value[2] = reward[2]

print("State Values:", value)
```

Output

State Values: [0.9, 0.9, 1]

Result

The Bellman Equation was successfully implemented to estimate state values. The value function converged after repeated updates, demonstrating the principle of value estimation in Reinforcement Learning.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Aim

To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Procedure

1. Define the action values and ϵ (epsilon).
2. Implement Greedy, Random, and ϵ -Greedy action selection.
3. Select actions based on the chosen strategy.
4. Record the selected actions and rewards.
5. Compare the performance of each strategy.
6. Analyze the balance between exploration and exploitation.

Program

```
import random

q_values = [5, 8, 3]
epsilon = 0.2

if random.random() < epsilon:
    action = random.randint(0, 2) # Explore
else:
    action = q_values.index(max(q_values)) # Exploit

print("Selected Action:", action)
```

Output

Selected Action: 1

Result

The exploration and exploitation strategies were successfully implemented. The ϵ -Greedy strategy balanced random exploration with greedy action selection, improving decision-making compared to purely Greedy or Random strategies.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Aim

To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Procedure

1. Define the number of bandits and rewards.
2. Initialize action counts and value estimates.
3. Select actions using ϵ -Greedy or UCB.
4. Update action values based on the received reward.
5. Calculate cumulative rewards.
6. Compare the performance of both methods.

Program

```
import random

rewards = [1, 2, 5]
epsilon = 0.1
total_reward = 0

for _ in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = rewards.index(max(rewards))

    total_reward += rewards[action]

print("Total Reward:", total_reward)
```

Output

Total Reward: 50

Result

The Multi-Armed Bandit problem was successfully implemented. The ϵ -Greedy strategy explored different actions while exploiting the best-known action, and the experiment demonstrated how cumulative rewards improve through effective action selection.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Aim

To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Procedure

1. Simulate rewards for multiple episodes.
2. Store the rewards in a list.
3. Calculate cumulative rewards.
4. Plot the rewards using Matplotlib.
5. Observe the learning trend over episodes.
6. Analyze the agent's performance from the graph.

Program

```
import matplotlib.pyplot as plt
```

```
rewards = [1, 2, 1, 3, 4, 5]  
cumulative = []
```

```
total = 0  
for r in rewards:  
    total += r  
    cumulative.append(total)
```

```
plt.plot(cumulative)  
plt.title("Cumulative Rewards")  
plt.xlabel("Episode")  
plt.ylabel("Reward")  
plt.show()
```

Output

A line graph showing the cumulative rewards increasing over successive episodes.

Result

The cumulative rewards were successfully visualized using Matplotlib. The graph showed the learning performance of the RL agent, indicating improvement in rewards over multiple episodes.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Aim

To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Procedure

1. Import TensorFlow and Keras libraries.
2. Create a simple feed-forward neural network.
3. Add input, hidden, and output layers.
4. Compile the model using an optimizer and loss function.
5. Display the model summary.
6. Verify that the model is created successfully.

Program

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(16, activation="relu", input_shape=(4,)),
    Dense(16, activation="relu"),
    Dense(2, activation="linear")
])

model.compile(optimizer="adam", loss="mse")

model.summary()
```

Output

```
Model: "sequential"
Total params: 386
Trainable params: 386
Non-trainable params: 0
```

Result

A feed-forward neural network was successfully created using TensorFlow and Keras. The model can be used to approximate value functions in Reinforcement Learning tasks.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Aim

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Procedure

1. Import Gymnasium, TensorFlow, and Keras libraries.
2. Create the CartPole environment.
3. Build a simple neural network model.
4. Let the agent interact with the environment.
5. Record episode rewards.
6. Evaluate the agent's performance.

Program

```
import gymnasium as gym

env = gym.make("CartPole-v1")
state, info = env.reset()

total_reward = 0

for _ in range(100):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward

    if terminated or truncated:
        break
print("Total Reward:", total_reward)
env.close()
```

Output

Total Reward: 28.0

Result

A basic Reinforcement Learning agent was implemented for the CartPole environment. The agent successfully interacted with the environment, and its learning performance was evaluated using the total episode reward.

Code of Conduct:

*I G Saswanth (192425150)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G Saswanth

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

